Methods Overview: Parsing, Chunking, and Evaluation

Same parser, same gold set — only the retrieval unit changes

Scope: 7 PDFs / 27 pages / 130 gold questions; parser `unstructured` `hi_res`; CPU, no paid API.

Data sources: `data/eval/three_methods.json`, `data/eval/colpali_results.json`, `data/eval/gold_pages.json`.

Outline

1. Overview
2. Row-JSON
3. Chipper
4. Dual-granularity

Four parts: context and overview → deep dive **Row-JSON** → deep dive **Chipper** → how to reproduce the experiments.

Problem and dataset

Problem

- Borderless numeric tables in Vietnamese financial/ETF reports.
- Gold question: “what is the value for ticker X?”
- Goal: compare retrieval units on one parser; the comparison isolates chunking.

Dataset

- 7 PDFs: 3 broker forecast reports + 4 SSI bulletins; 27 pages; 10 gold tables.
- 130 questions, 10–40 per file; PDFs have a text layer, not scans.
- Parser-independent page gold: 111 one-page, 18 two-page, 1 three-page.

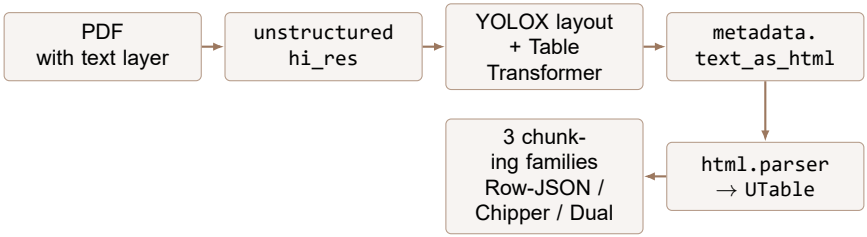
Parser coverage — read before the results

`unstructured` keeps **126/130 = 96.9%** of values at element level; **122/130 = 93.8%** at chunk level. All four misses are missing cell values, not whole rows.

Evaluation: chunk-level BM25 (ranking within each file); page-level max-pool over 27 pages; Recall@k + MRR; CPU only.

Gold QA comes from the archived parser’s table cells and is kept fixed to measure data lost when switching parsers.

What parses the PDF: one pipeline



Parse call

```
partition_pdf(strategy="hi_res",
infer_table_structure=True,
languages=["vie"])
```

Key points

- Tables come from `text_as_html`, not coordinates or ruling lines.
- `PyMuPDF` is used only for page gold and ColPali rendering.
- `row_json` does not parse PDFs; it only consumes `UTable`.

From table HTML to `UTable`

Four normalization steps

1. `html.parser.HTMLParser` reads `<tr>`/`<td>`.
2. Expand `colspan`/`rowspan` into a rectangular grid.
3. Header: prefer `<thead>`/`<th>`; otherwise infer from the numeric-cell ratio; join levels with `" / "`.
4. Drop trailing empty columns; attach the nearest preceding title/caption.

Shared output

```
UTable(
  columns=["Ticker", "Weight"],
  rows=[["MBB", "7.18%"], ...],
  page=2, caption="Table 1: ...")
```

Why this step matters

All three chunking families receive the same `UTable`. The only difference left is how the table is split, not how the PDF was parsed.

Warning: Table Transformer header/merge errors flow straight into JSON keys; the 4/130 missing values are a parser limit, not a chunking bug.

Methods used

Family	Splitting rule	#chunks	Returned unit	Best for
Row-JSON	one table row = one JSON object	332	one row	cell lookup
Chipper	4 rules; whole table = 1 chunk	136	2k chunk / whole table	table context
Dual	small sentences/rows + large passages	935	large passage or row	small2big
BM25 page	`unstructured` text, no chunking	27	page	baseline
ColPali	page image, zero-shot	27	page	page triage / scans

- All three text methods share **809 elements and 14 Table elements** (10 gold tables); chunk counts differ because the units differ.
- `chipper_prefix` is a Chipper variant: add the first 2 sentences or the caption to the chunk; no LLM.

Only the splitting unit changes; parser and retriever stay fixed.

Methods: pros / cons and suitable questions

Method	Pros / cons	Suitable questions
Row-JSON	**+** high precision, cheap, self-contained, easy to audit. − loses whole-table context; cannot aggregate across rows.	“What is MBB’s weight?”; compare a few tickers; one-cell lookup.
Chipper	**+** simple, parameter-free, preserves tables, high R@5/R@10. − whole table becomes one chunk; ∼2.3k chars; low R@1.	“What does this table say?”; summarization; LLM context.
Dual	**+** two levels, small2big, reranking. − many parameters; rerank/dense hurt cell precision.	Questions needing passage context; generation; prose.
ColPali	**+** no parser/OCR, sees layout. − page unit; no cell answer; 256M zero-shot is weak.	Scanned PDFs; page selection; fallback when the text layer is lost.
BM25 page	**+** cheap, easy baseline. − ignores table structure; very weak here.	Control baseline.

No universal best method: the unit must match the question type.

Results

Method	#chunks	R@1 chunk	R@5 chunk	MRR chunk	R@1 page	MRR page
Row-JSON	332	**81.5%**	**90.0%**	**0.850**	**90.0%**	**0.950**
`dual_small`	935	66.1%	78.5%	0.723	73.9%	0.844
`dual`	935	50.0%	68.5%	0.583	56.1%	0.751
`chipper_prefix`	136	36.9%	77.7%	0.535	45.4%	0.665
`dual_dense`	935	31.5%	63.8%	0.441	44.6%	0.700
`chipper`	136	27.7%	76.9%	0.475	36.1%	0.618
ColSmol-256M	–	–	–	–	13.8%	0.288
BM25 page	–	–	–	–	4.6%	0.227

Sorted by R@1 chunk. #chunks applies to text methods only; page-level max-pool over 27 pages, not 27 chunks.

- **Row-JSON** beats `dual_small` by 15.4 points R@1 chunk and is $\sim 3\times$ Chipper; page R@1 reaches 90.0%.
- **Chipper** has R@10 89.2% but R@1 27.7%: it *contains* the answer but does not rank the cell first.
- **ColPali** 13.8% page $>$ BM25 4.6%, but is still $\sim 6.5\times$ below Row-JSON.

Choose by goal

Goal	Choose	Why
Single-cell value / one ticker	**Row-JSON** + BM25	81.5% chunk R@1, 90.0% page R@1; ∼491 chars per chunk.
Need whole-table context	Row-JSON + parent table, or Chipper	Keep small2big without diluting the cell index.
Qualitative / prose	Dual (large) or a separate prose index	Avoid table chunks taking top-1 on numeric questions.
Scanned PDF / no text layer	ColPali after fine-tuning, with GPU	256M zero-shot gives only 13.8% page R@1; not enough to answer.
Academic baseline	Chipper (+ prefix) / BM25 page	Parameter-free, reproducible, easy to compare.

Cross-cutting lesson

Do not mix all chunk types in one index: BM25 cannot distinguish chunk “contains” from chunk “answers”. Adding prose or whole tables to a numeric index drops R@1 sharply (`dual` 50.0%, `chipper` 27.7%).

Deep dive 1

Row-JSON

`row_json` — one table row = one JSON object

Guiding question

Why does “one row = one chunk” win on cell questions?

Row-JSON: one table row, one object

One row — one self-contained object

```
{"file": "VN30 bulletin Q2/2026",
  "page": 2,
  "table": "Table 1: ...",
  "row": {"Ticker": "MBB",
        "New weight": "7.18%"}}
```

Illustrative; real keys come from the table header (often Vietnamese in this corpus).

Why it works

- Entity + value live in **the same chunk**.
- Header is repeated → BM25 matches column names.
- File/page/caption are self-contained metadata.
- Deterministic, no LLM, no GPU; easy to audit.

Naming

Slides call it **Row-JSON**; code calls it `row_json`. This is *not* JSONL: one **table row** is one object, not one file line.

332 chunks across 7 documents; ∼491 chars per chunk. Only rows with at least one number become chunks.

Path of one row

From PDF to chunk

1. `parse_unstructured.py` → cache of elements + HTML.
2. `uelements.parse_table_html` → `UTable`.
3. `row_json_chunks` iterates `table.rows`.
4. `table_row_json` → JSON for each row.

Use as a library

```
from etflab.uelements import load_unstructured_doc
from etflab.chunking_methods import row_json_chunks

doc = load_unstructured_doc(path)
chunks = row_json_chunks(doc, doc["elements"])

print(chunks[0].text)
```

`row_json` does not read PDFs or use coordinates; it only consumes the reconstructed table.

- Rows without numbers are skipped; header/empty rows create no chunk.
- No table splitting, no row merging: one row = one event.
- If `text_as_html` is empty, there is no HTML table → no chunks.

Row-JSON: pros / cons

Pros

- Highest R@1 among text methods: 81.5% chunk / 90.0% page.
- Small chunks ($\sim$491 chars), focused signal.
- Self-contained context: file, page, caption, column names.
- No model, no GPU; low ingest cost.

Cons

- Loses global table structure: multi-row questions need a parent table.
- Rows without numbers are dropped; descriptive rows may disappear.
- Parser header/cell errors flow straight into JSON keys.
- Does not answer qualitative prose questions.

Strongest point: a row contains entity, value, and metadata. **Weakest point:** no whole-table context.

Row-JSON: suitable questions

Use when

- “What is MBB’s weight?”
- “Compare the weights of X and Y” (top-k rows).
- “Value for Q2.”
- “Is this ticker in the basket?”

Do not use alone

- “Sum of five tickers.”
- “Trend across the whole table.”
- “Why did the index change?”

For multi-row questions: retrieve top-k rows, then expand to the parent table or a prose chunk. Do not expect one row to answer an aggregation.

Row-JSON: results and limits

Results

	Chunk	Page
R@1	81.5%	90.0%
R@3	90.0%	100%
R@5	90.0%	100%
R@10	90.0%	100%
MRR	0.850	0.950
chars@1	494	–

Why does it stop at 90%?

- Coverage 93.8% (122/130): 4 values missing in parser output, 4 entity/value pairs split across chunks.
- R@3 = R@10 = 90.0%: 8 questions outside coverage, 5 have an answer but are outside top-3.
- 4 misses: TCB 8.75%, TPB 3,729, MBB 65,517, MBB 24.4.

To go beyond 90%

Fix the parser (header/merge) or add parent-table expansion. Tuning BM25 cannot recover the 4 values absent from parser output.

Deep dive 2

Chipper

element-based chunking, arXiv:2402.05131

Guiding question

Why does a method that follows the paper still underperform on numeric tables?

Chipper: model or method?

Two things often confused

Chipper is the name of a *layout model* (Unstructured's vision encoder-decoder). The **method** in the paper is **element-based chunking**. This project uses the *method*, not the Chipper model.

Paper

- Four rules, no parameters; 2,048-char threshold.
- FinanceBench (10-K/10-Q), English.
- Goal: "best chunk size without tuning".

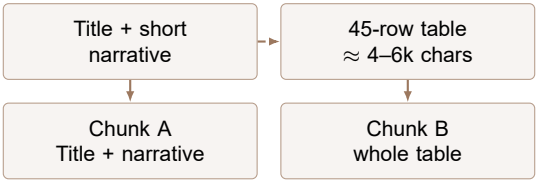
Project

- Same `unstructured` element stream.
- 136 chunks; tables kept whole.
- `chipper_prefix`: add first 2 sentences / caption.
- No GPT-4, no API.

This is a reproducible baseline, not a comparison of the Chipper model.

Chipper's four rules

1. **Follow the reading order** of elements.
2. **Pack up to 2,048 chars**; never split an element.
3. **Title** $\rightarrow$ start a new chunk (title stays in the new chunk).
4. **Table** $\rightarrow$ start a new chunk and **keep the whole table**.



Consequence

A 45-row table remains **one chunk**, even far above 2,048 chars. That is rule 4, not an implementation bug. Chipper chunks are often "the whole table", so BM25 has one coarse signal.

Application and pros / cons

Application on `unstructured`

- Elements: title, narrative, list, table, caption.
- Skip header/footer/pagebreak/image/formula.
- `chipper_prefix`: first 2 sentences (prose) or caption (table).
- 136 chunks; average 557 chars (prefix: 662).

Pros

- Simple, parameter-free, reproducible.
- Keeps tables intact; high R@5/R@10.
- Broad context, good for generation.

Cons

- Whole-table chunk $\sim$2.3k chars, low R@1.
- Large tables far exceed 2,048.
- Caption/header are not enough to locate a cell.

Suitable questions

"What does this table say?" · summarization · LLM context. **Not:** "What is MBB's weight?".

Chipper: results

Variant	#chunks	avg chars	R@1	R@5	R@10	MRR	R@1 page
`chipper`	136	557	27.7%	76.9%	89.2%	0.475	36.1%
`chipper_prefix`	136	662	36.9%	77.7%	89.2%	0.535	45.4%

`chars@1`: 2,299 (chipper) and 2,430 (prefix) — $\sim$4.7–4.9$\times$ Row-JSON. R@10 nearly matches Row-JSON, but R@1 is only $\sim$1/3.

- Prefix improves **+9.2 points R@1 chunk** and **+9.3 points R@1 page**; it is still 44.6 points behind Row-JSON.
- R@10 89.2% is evidence of "contains" but not "answers": the table is in top-10, rarely first.
- Typical failures: 45-row tables; lost captions; adjacent tables merged; numbers in headers confuse BM25.

Key takeaway

Chipper is a context builder, not a cell locator.

Deep dive 3

Dual-granularity

`dual_small`, `dual`, `dual_dense` — small-to-big

Guiding question

Why does returning the **whole table** hurt cell lookup?

Dual-granularity: small-to-big idea

Two levels from same elements

- **Large** (`dual\_large`): prose grouped by `title` up to `large\_target`; **each table becomes a single large chunk**.
- **Small**: derived from large. For numeric table rows -> `dual\_row` (with `parent\_id`); for prose -> sentences/groups.
- Linked by `parent\_id`.

Small-to-big retrieval

- **Search on small**: find precise location.
- **Expand to parent**: return large chunk for context.
- Balance **precision** (small) + **context** (large).

Same retriever

Only **indexed set** and **returned chunk** differ. BM25 + doc-filter unchanged.

dual_small vs dual vs dual_dense

Variant	Index / Return	Description
dual_small	small only / small	Search dual_row + dual_small. Return small chunk. Best for “value of ticker X”.
dual	small / large (parent)	Find on small, expand to dual_large. For tables this is **the whole table**.
dual_dense	small / large + e5-small	Same as dual, with sentence merging + dense rerank. Helps prose, hurts rigid numeric tables.

Why returning the whole table hurts

Cell queries

- `dual\_small`: hits exact `dual\_row` with ticker+value.
- `dual`: must rank inside a 45-row table -> diluted.
- `dual\_dense`: reranking favors broader semantic matches.

Chunk-level (130 questions)

Variant	#chunks	R@1	R@5	R@10	MR
`dual\_small`	935	66.1%	78.5%	86.2%	0.7
`dual`	935	50.0%	68.5%	68.5%	0.5
`dual\_dense`	935	31.5%	63.8%	63.8%	0.4

Takeaway

Small-to-big is good when parent adds context. Bad when parent = entire table.

Dual vs Row-JSON vs Chipper

When to use

- **Prose/narrative**: explanations benefit from context.
- **Mixed docs**: need both small precision and big context.
- **Hybrid strategy**: put table cells in tight units; keep prose in Dual if needed.

Chunk-level R@1

Method	R@1
`row\_json`	**81.5%**
`dual\_small`	66.1%
`dual`	50.0%
`chipper\_prefix`	36.9%
`dual\_dense`	31.5%
`chipper`	27.7%

Bottom line

For “value of ticker X”, **keep unit small**. `row\_json` best overall; `dual\_small` second among dual; returning whole table hurts badly.